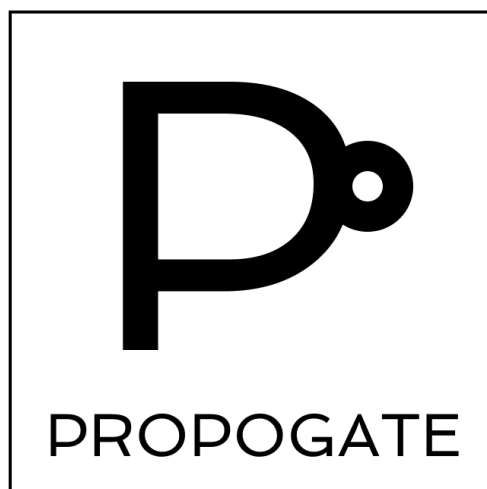




Propagate

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v1.0.0

1. Equipo	1
2. Repositorio	1
3. Introducción	2
4. Modelo Computacional	2
Dominio	2
Lenguaje	2
5. Implementación	2
Frontend	2
Backend	2
Dificultades encontradas	2
6. Futuras extensiones	2
7. Conclusiones	2
8. Referencias	2
9. Bibliografía	2

1. Equipo

Nombre	Apellido	Legajo	E-mail
Mateo Hernán	Cornejo	62.722	macornejo@itba.edu.ar
Ignacio Gastón	Frund	61.465	ifrund@itba.edu.ar

2. Repositorio

La solución y su documentación se encuentran versionadas en: [Propogate](#).

3. Introducción

La forma de escribir fórmulas en la lógica proposicional cuenta con un grado de simplicidad que provoca que su lectura resulte densa y confusa, dificultando su seguimiento y comprensión. Para simplificar dicha lectura, se propone como solución un lenguaje específico de dominio (DSL) que interprete fórmulas escritas en el modo de matemáticas de un archivo LaTeX y las traduzca a circuitos de compuertas lógicas, sin modificar el resto del archivo.

4. Modelo Computacional

Dominio

Las fórmulas y variables se definen en modo de matemáticas de LaTeX, respetando la nomenclatura de la lógica proposicional. La salida también es un archivo LaTeX, que genera los diagramas utilizando el paquete [circuitikz](#).

Lenguaje

Para indicar que se desea utilizar el lenguaje, se utiliza el comando `[!propogate]{ ... }` dentro del modo de matemáticas, y dentro de las llaves utilizando la sintaxis correcta:

Concepto	Sintaxis	Ejemplo
Fórmula unaria	<code>[¬] {(FORMULA) VARIABLE }</code>	<code>\$[!propogate]{ \neg (\alpha) }\$</code>
Fórmula binaria	<code>(FORMULA {^ V →} FORMULA)</code>	<code>\$[!propogate]{ \alpha \wedge \beta }\$</code>
Definición de variable	<code>VARIABLE = {true false}</code>	<code>\$[!propogate]{ p = true }\$</code>
Definición de fórmula	<code>formname = FORMULA</code>	<code>\$[!propogate]{ \alpha = p }\$</code>

El lenguaje permite, además, definir una serie de expresiones (fórmulas o definiciones) separadas por coma , en un mismo entorno del comando.

Por claridad, las variables corresponden a una secuencia de letras latinas minúsculas mientras que las fórmulas a una secuencia de letras griegas minúsculas.

A su vez, el proyecto incluye un poco de parseo de LaTeX para asegurar el correcto procesamiento del documento, sobre el cual no se entra en detalle por exceder los propósitos del lenguaje definido.

5. Implementación

Frontend

Flex

Para el desarrollo de Flex inicialmente se buscó que se controlen las situaciones de LaTeX (encontrarse en modo de matemática, esté definido el tipo de documento, entre otros) dentro de los mismos patrones, pero esto se modificó tras correcciones.

La versión final busca analizar principalmente lo que ocurre una vez se especificó el comando del proyecto (contexto **PROPOGATE**), y algunas palabras clave puntuales de LaTeX para garantizar el uso correcto del modo de matemática y de la definición del archivo.

Principalmente se distinguen fórmulas unarias y binarias, nombres de variables y fórmulas, valores de verdad, el operador de igualdad (definición) y el separador, además del comando y el símbolo que finaliza su contexto.

Bison

En el caso de Bison y la gramática en general, lo que solía controlarse con Flex requirió la incorporación de muchas reglas exclusivas a LaTeX por encima de las pertinentes al proyecto. Sin embargo, esto permite una sintaxis más general en los archivos LaTeX de entrada.

Sobre el proyecto, se busca respetar las reglas de la lógica proposicional: las variables pueden tener valor de verdad, las fórmulas deben tener paréntesis y pueden definirse recursivamente, y se utiliza una lista para controlar los distintos tipos de expresiones.

Backend

Específico del dominio

La definición del dominio se basó en terminar de definir el frontend para poder trabajar a partir del árbol de sintaxis abstracta (AbstractSyntaxTree, AST). Luego fue necesario crear una función que recorriera la totalidad del AST de forma recursiva, una lista de listas a grandes rasgos de acuerdo a la definición de la gramática. También se realizó la tabla de símbolos, para poder interpretar fórmulas anidadas complejas; en este momento se observó la necesidad de asignarle un valor por defecto a la declaración de variables porque, de no tener valor, ya el primer test era rechazado. Se optó por utilizar la equivalencia lógica $(A \rightarrow B) = (\neg A \vee B)$ para evaluar el operador binario de implicación.

Generación de código

En un principio fue sencillo adaptar la lógica del proyecto original y de otros trabajos relativos al uso de LaTeX como referencia. Se identificó la librería para utilizar compuertas lógicas en LaTeX, con la cual se evaluaron nuevamente los casos de uso del lenguaje para generar un subset mínimo que nos permitiese obtener la salida esperada. También fue imperativo considerar que Propagate se encontrase o no envuelto en contexto de LaTeX y, de ser así, si se

encontraba en modo de matemática para poder respetar los símbolos y su orden. Para ello se modificó la gramática con tal de que los saltos de línea persistan en tokens. Se descubrió la necesidad de realizar un recorrido recursivo por los elementos, tanto para detectar envoltorio de código en forma de LaTeX y para poder dibujarlos manteniendo el orden. Se optó por modificar el comando `test.sh` con tal de que el nombre del archivo sea dinámico según el archivo de entrada, de modo tal que al ejecutarlo se generan tantos archivos de salida como archivos de prueba.

Dificultades encontradas

En el frontend, se solicitó cambiar la implementación donde se detectaba el correcto uso de LaTeX mediante patrones de Flex por reglas de la gramática, lo cual conllevó a una gran redefinición que limitó mucho a los patrones y aumentó considerablemente la complejidad de la misma, además de volverla híbrida cuando antes era sólo pertinente al proyecto. A su vez, se requirió de paciencia para controlar correctamente los casos donde el documento entra y sale del modo de matemática, lo cual se consideró de importancia para que las fórmulas se representen correctamente en el documento LaTeX compilado.

Estos cambios de complejidad se arrastraron a la generación de código, donde se debieron controlar muchísimos más casos para mantener una estructura válida en la salida que no pierda información.

6. Futuras extensiones

Estas son algunas de las extensiones que estaban planificadas para esta entrega pero no se encuentran disponibles aún:

- Reutilizar fórmulas y variables y que las mismas conserven su previa definición.
- Distinguir entre las definiciones de fórmulas y variables según la sección del documento LaTeX, permitiendo que pueda reutilizarse el mismo nombre de fórmula o variable con otro significado (scope).
- Utilizar compuertas lógicas que no sean literales de la lógica proposicional para armar circuitos complejos de forma compacta (por ejemplo, utilizar compuertas XOR).
- Enumerar y describir los distintos circuitos para facilitar su referencia dentro del documento.

7. Conclusiones

Afrontar una problemática como esta desde el enfoque del diseño e implementación de un lenguaje simplifica la lógica y permite una solución más poderosa que la alternativa de desarrollar un programa puntual que cumpla la misma función, aunque requiera incorporar más conceptos y contemplar muchísimos más casos para cumplir con la generalidad que exige.

Además, se considera que la premisa del proyecto es limitada y apunta a una problemática muy específica que al fin y al cabo no es de mucha gravedad, por lo que se considera este proyecto más como una herramienta de aprendizaje que un producto.

8. Bibliografía

1. Agustín Golmar et al, "Flex-Bison-Compiler," *Github*. 2025, Septiembre 3. [En línea]. Disponible en: <https://github.com/agustin-golmar/Flex-Bison-Compiler/tree/v2.0.0>
2. The Flex Project, "Lexical Analysis With Flex, for Flex 2.6.2," *Github Pages*. 2025, Octubre 19. [En línea]. Disponible en: <https://westes.github.io/flex/manual>
3. Free Software Foundation, "GNU Bison - The Yacc-compatible Parser Generator," *GNU Org*. 2025, Octubre 19. [En línea] Disponible en: <https://www.gnu.org/software/bison/manual>
4. Overleaf, "Mathematical expressions - Overleaf, Online LaTeX Editor," *Overleaf*. 2025, Octubre 20. [En línea]. Disponible en: https://www.overleaf.com/learn/latex/Mathematical_expressions
5. Overleaf, "CircuiTikz package - Overleaf, Online LaTeX Editor," *Overleaf*. 2025, Septiembre 3. [En línea]. Disponible en: https://www.overleaf.com/learn/latex/CircuiTikz_package
6. M. A. Redaelli et al, "CircuiTikZ 1.8.3 - manual", *uchile.cl*. 2025, Noviembre 28. [En línea] Disponible en: <https://ctan.dcc.uchile.cl/graphics/pgf/contrib/circuitikz/doc/circuitikzmanual.pdf>
7. M. E. Zahnd et al, "apuntes-9335-logica-computacional," *Github*. 2025, Septiembre 3. [En línea] Disponible en: <https://github.com/mzahnd/apuntes-9335-logica-computacional>